arXiv:2610.19928v1 [cs.CL] 14 Oct 2026

MemDuet: Learning Dual-Process Memory for Long-Context Reasoning

Jiangcheng Song[1], Haodong Jing[1], Yulin Mu[2], Lesong Tao[1], Jixin Wang[1]
Jin Cui[1], Tianjing Ren[3], Runxuan Xie[2], Yongqiang Ma[1], Nanning Zheng[1]*
[1]State Key Laboratory of Human-Machine Hybrid Augmented Intelligence,
Institute of Artificial Intelligence and Robotics,
Xi'an Jiaotong University
[2]Faculty of Electronic and Information Engineering,
Xi'an Jiaotong University
[3]Imperial College London
enone@stu.xjtu.edu.cn, nnzheng@xjtu.edu.cn

Abstract

Reasoning over infinitely long documents is becoming a reality with recurrent memory agents. However, increasing context length exacerbates the trade-off between reasoning performance and inference efficiency. Inspired by Kahneman's account of fast and slow thinking, we propose **MemDuet**[1], a dual-process memory framework that learns to coordinate a small language model for fast, parallel evidence selection from raw context chunks (System 1) with a large language model for memory-update verification and memory updates in document order (System 2). We train MemDuet in three stages: supervised fine-tuning of System 1 for evidence selection, Group Relative Policy Optimization (GRPO) of System 2 for verification and memory rewriting, and joint optimization of both policies to coordinate selection and updating for long-context reasoning. Experiments on diverse long-context reasoning benchmarks (RULER-HQA, LongBench v2, and 2WikiMultiHopQA) demonstrate a favorable accuracy–efficiency trade-off as context length grows. Averaged across evaluation settings on the three datasets, MemDuet achieves an end-to-end inference speedup of **9.74**× and an output-token reduction of **73.91%** relative to RL-MemAgent.

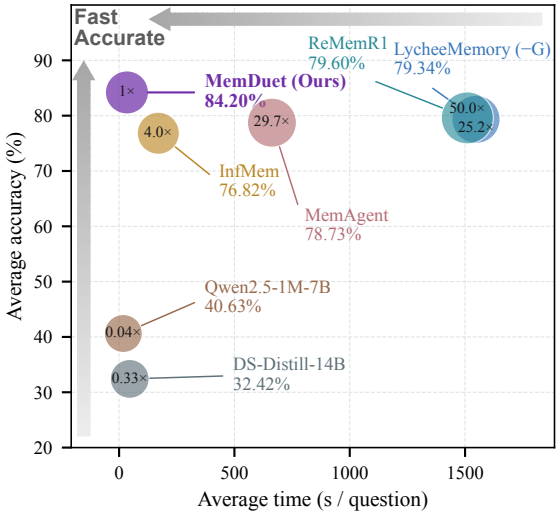


Figure 1: Accuracy–efficiency trade-off on RULER-HQA. Accuracy, inference time, and output tokens are equally averaged over the nine shared length settings (7K–1.75M). Qwen and DS-Distill use the available settings. Circle areas use a square-root-compressed scale for mean output tokens per question; circle labels give actual output-token ratios to MemDuet (1×).

1 Introduction

Long-context reasoning is essential for large language models (LLMs) to synthesize evidence across documents, understand lengthy reports, and resolve repository-level dependencies in code (Bai et al., 2024; Zhang et al., 2023). Existing work primarily extends long-context capability through larger context windows, sparse or linear attention, and retrieval-augmented generation (RAG) (Chen et al., 2023b; Beltagy et al., 2020; Katharopoulos et al., 2020; Yuan et al., 2025; Lewis et al., 2020). However, these approaches remain bounded by the amount of context that can be directly represented or retrieved, leaving reasoning over unbounded documents unresolved.

MemAgent instead streams documents through a bounded recurrent memory, enabling reasoning over infinitely long documents with a fixed active context (Yu et al., 2026). Building on this paradigm, InfMem and GRU-Mem learn when to retrieve, update, or stop, while LycheeMemory compresses

*Corresponding author: nnzheng@xjtu.edu.cn.
[1]Project page: jiangchengsong.github.io/MemDuet/.

each chunk and invokes the reasoner only for selected memory blocks (Wang et al., 2026; Sheng et al., 2026; Chen et al., 2026). These advances make recurrent memory increasingly selective, but expose a common bottleneck: control decisions either remain inside the sequential recurrent loop or operate on representations in which the original evidence has already been compressed. Consequently, as context grows, reasoning performance and inference efficiency remain tightly coupled.

Human reading and memory suggest a different principle: memory formation begins with selective attention. Readers first focus on goal-relevant information and then integrate the attended evidence with prior knowledge into a coherent memory representation (Craik et al., 1996; Kintsch, 1988; Aly and Turk-Browne, 2016). This process naturally maps onto the dual-process account of cognition: **System 1** rapidly identifies potentially useful evidence, whereas **System 2** deliberately assesses and consolidates it (Kahneman, 2011; Evans and Stanovich, 2013). Based on this insight, we propose **MemDuet**, a dual-process recurrent memory framework. A small language model (System 1) scans raw context in parallel to select candidate chunks for memory updating. A large language model (System 2) verifies whether each selected chunk warrants a memory update under the current memory, then rewrites memory. By combining small-model selective attention over raw chunks with large-model verification and memory updating, MemDuet balances reasoning performance and inference efficiency as context grows.

A key challenge is how to train the two systems to cooperate. We develop a progressive three-stage training strategy. First, we supervise System 1 to identify context chunks that are likely to trigger useful memory updates. Next, with System 1 fixed, we optimize System 2 using Group Relative Policy Optimization (GRPO; Shao et al., 2024), training it to verify memory-update decisions and rewrite memory toward successful downstream reasoning. Finally, we jointly optimize both systems with task-level rewards, allowing evidence selection and memory updating to co-adapt.

We evaluate MemDuet on RULER-HQA, LongBench v2, and long-context 2WikiMultihopQA. Results show that MemDuet improves the performance–efficiency trade-off over full-update recurrent memory agents, scaling to 3.5M-token contexts while achieving higher reasoning accuracy with substantially lower inference cost.

Our contributions are summarized as follows:

1. We propose MemDuet, a cognitively inspired dual-process memory framework for long-context reasoning, which instantiates System 1 as fast, parallel evidence selection with a small language model and System 2 as memory-update verification and memory rewriting with a large language model.

2. We develop a three-stage training strategy that first establishes the selection and memory-updating capabilities of the two systems through SFT and GRPO, respectively, and then jointly optimizes their cooperation under a shared downstream reasoning objective.

3. Extensive experiments on long-context reasoning benchmarks demonstrate that MemDuet achieves a favorable balance between reasoning performance and inference efficiency.

2 Related Work

2.1 Recurrent Memory for Long-Context Reasoning

Recurrent-memory methods process documents incrementally while maintaining a bounded state, allowing language models to handle inputs far beyond their context windows. MemAgent sequentially processes text chunks and repeatedly rewrites a textual memory, using end-to-end reinforcement learning to retain task-relevant information and discard distractors based on downstream rewards (Yu et al., 2026). Building on this paradigm, InfMem and GRU-Mem learn retrieval, memory updating, and early-stopping decisions through SFT-to-RL training or behavior-specific rewards (Wang et al., 2026; Sheng et al., 2026). LycheeMemory compresses individual chunks and selectively invokes the reasoner, jointly optimizing its compressor and reasoner through end-to-end reinforcement learning (Chen et al., 2026). These studies turn memory management from a fixed procedure into a task-driven policy, but their control decisions remain within the sequential loop or operate on already compressed evidence. MemDuet instead scans raw chunks in parallel before recurrent updating and progressively coordinates evidence selection with memory consolidation.

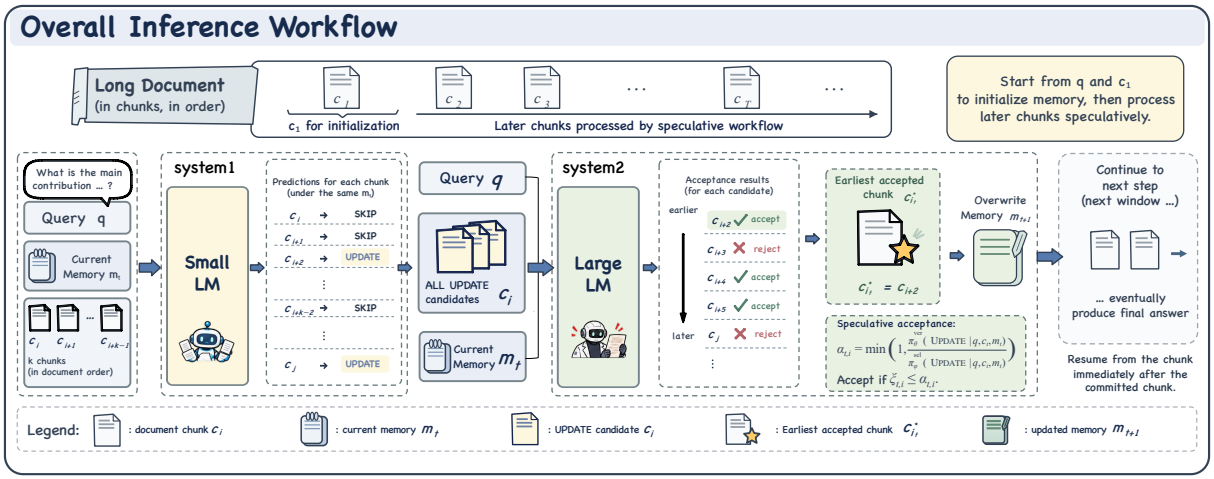


Figure 2: MemDuet's overall inference workflow. System 1 predicts SKIP/UPDATE in parallel for raw chunks under a shared memory state. Candidate chunks retain their original text. System 2 verifies whether a selected chunk warrants a memory update given the current memory, then generates and commits memory from the first accepted chunk in document order. Later chunks are reevaluated under the updated memory.

2.2 Cognitively Inspired Language Models and Memory

Cognitive theories have informed both memory design and computational allocation in language-model systems. CoALA organizes language agents around modular memory, decision-making, and action components derived from cognitive architectures (Sumers et al., 2024). MemoryBank adopts an Ebbinghaus-inspired mechanism for dynamically forgetting and reinforcing memories (Zhong et al., 2024), whereas HippoRAG draws on hippocampal indexing theory to support associative knowledge retrieval (Gutiérrez et al., 2024). For reasoning, System 2 Attention deliberately rewrites the context to remove irrelevant information (Weston and Sukhbaatar, 2023), while Dualformer learns switchable fast and slow reasoning modes within a single model (Su et al., 2025). These studies primarily apply cognitive principles to memory organization, retrieval, or the allocation of reasoning effort. MemDuet instead introduces dual-process specialization into recurrent memory: System 1 selects raw candidate chunks in parallel; System 2 verifies whether they warrant memory updates under the current memory and consolidates their evidence.

3 Methodology

3.1 Preliminaries

Problem setup. We consider query-conditioned reasoning over a long document D. Given a query q, the goal is to generate an answer y using evidence from D, whose length may exceed the model's context window. We divide D into T chunks $(c_1, \ldots, c_T)$ in their original order, with $|c_i| \leq C$, where $|\cdot|$ denotes token length and C is the chunk-size budget. The query is available throughout document processing.

Recurrent textual memory. Following MemAgent (Yu et al., 2026), an LLM carries information across chunks through bounded textual memory, using the common interface

$$m' \sim \pi_\theta^{\text{mem}}(\cdot \mid q, c_i, m), \qquad |m'| \leq B, \quad (1)$$

Here m and m' are the preceding and replacement memories, and B is the memory budget. Under the memory prompt, π_θ^{mem} is the autoregressive product of System 2's next-token policy π_θ^{gen}. The same model answers from q and the final memory under the answer prompt. Both C and B are independent of document length.

3.2 MemDuet Workflow

MemDuet alternates parallel raw-chunk selection by System 1 with verification and memory updates in document order by System 2 (Figure 2).

3.2.1 System 1: Memory-Conditioned Parallel Candidate Selection

Here t indexes speculative rounds and i document chunks. At round t, System 1 independently evaluates a window $\mathcal{W}_t$ of up to k consecutive chunks in parallel, using the shared query q and memory m_t

to select candidate chunks for memory updating:

$$a_{t,i} \sim \pi_\phi^{\text{sel}}(\cdot \mid q, c_i, m_t), \quad a_{t,i} \in \{\textsc{Skip}, \textsc{Update}\}. \tag{2}$$

The chunks predicted as UPDATE form the candidate set

$$\mathcal{C}_t = \{c_i \in \mathcal{W}_t \mid a_{t,i} = \textsc{Update}\}, \tag{3}$$

which is forwarded to System 2.

3.2.2 System 2: Speculative Verification and Memory Consolidation

Inspired by speculative decoding's acceptance rule (Leviathan et al., 2023; Chen et al., 2023a), System 2's binary policy π_θ^{ver} assesses whether selected chunks warrant memory updates. For candidate indices $\mathcal{I}_t = \{i \mid c_i \in \mathcal{C}_t\}$, the action-level acceptance probability is

$$\alpha_{t,i} = \min\left(1, \frac{\pi_\theta^{\text{ver}}(\textsc{Update} \mid q, c_i, m_t)}{\pi_\phi^{\text{sel}}(\textsc{Update} \mid q, c_i, m_t)}\right). \tag{4}$$

Independently draw $\xi_{t,i} \sim \mathcal{U}(0,1)$ for each candidate and accept it if $\xi_{t,i} \le \alpha_{t,i}$.

If any candidate is accepted, commit the first accepted chunk in document order and generate its replacement memory:

$$m_{t+1} \sim \pi_\theta^{\text{mem}}(\cdot \mid q, c_{i_t^\star}, m_t), \tag{5}$$

where $i_t^\star = \min\{i \in \mathcal{I}_t \mid \xi_{t,i} \le \alpha_{t,i}\}$. As a special case, if no candidate is accepted, retain the current memory and advance past the window. Otherwise, discard remaining predictions conditioned on m_t and resume immediately after the committed chunk with m_{t+1}. Answering uses the query and final memory; Algorithm 3 in Appendix B.4 gives the complete transitions.

3.3 Training of MemDuet

Figure 3 summarizes three-stage training. Stage I trains memory-conditioned evidence selection. Stage II freezes System 1 and trains System 2 to verify candidates, rewrite memory, and generate answers. Stage III jointly optimizes both systems with task rewards, coordinating selection with verification and memory updates.

3.3.1 Stage I: System 1 Supervised Warm-up

We initialize System 1 on HotpotQA (Yang et al., 2018) by aligning gold supporting facts with document chunks. Evidence-containing chunks are labeled UPDATE, and other chunks SKIP.

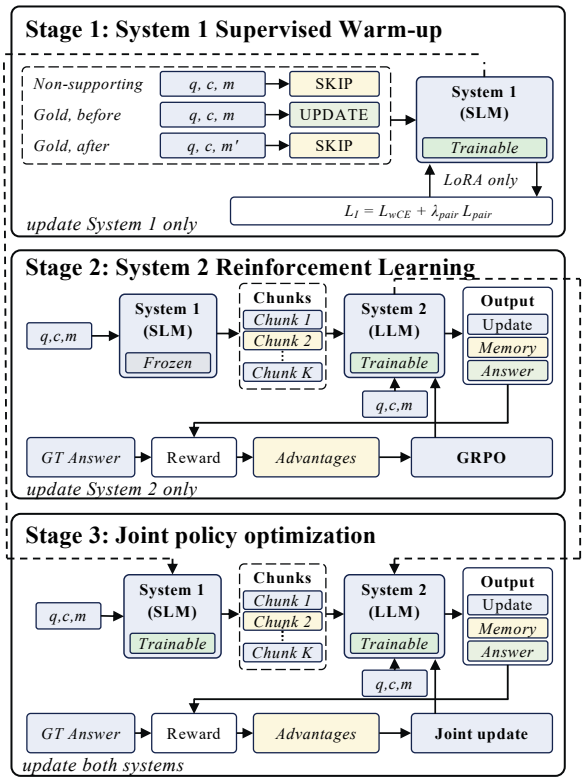


Figure 3: Three-stage training of MemDuet. Stage I trains System 1 with weighted classification and a paired-memory hinge loss. Stage II freezes System 1 and optimizes System 2 with GRPO. Stage III jointly optimizes both systems using task rewards.

For each UPDATE example (q, c, m), let m' be the memory after incorporating the evidence from c. We retain the original example and add a post-update negative:

$$(q, c, m) \longrightarrow \textsc{Update}, \quad (q, c, m') \longrightarrow \textsc{Skip}. \tag{6}$$

We train on the original examples and post-update negatives using the memory-contrastive objective:

$$\mathcal{L}_{\text{Stage I}}(\phi) = \mathcal{L}_{\text{wCE}} + \lambda_{\text{pair}}\mathcal{L}_{\text{pair}}, \quad \mathcal{L}_{\text{pair}} = \mathbb{E}_{\mathcal{P}}\left[\max(0, \gamma - \Delta s_\phi)\right]. \tag{7}$$

Here, $\mathcal{L}_{\text{wCE}}$ is class-weighted action cross-entropy; $\mathcal{P}$ contains matched memory pairs, with $\Delta s_\phi = s_\phi(q, c, m) - s_\phi(q, c, m')$. The score s_ϕ is the UPDATE-minus-SKIP logit difference, and γ is the margin. Hyperparameters are reported in Appendix B.1.

3.3.2 Stage II: System 2 Reinforcement Learning

We freeze System 1 at ϕ_{SFT} and optimize all System 2 parameters θ by reinforcement learning over